HWDC 2026

產出的人不能驗自己

AI 時代的驗收設計

— 我的 SDD＋Harness 開發藍圖

楊騏 Ci

WaCare 數位健康 · Tech Lead

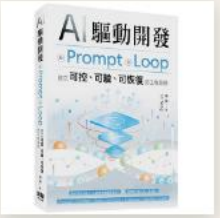




楊騏（Ci）

我是 Ci，專注讓 AI 驅動開發在團隊落地

現職	WaCare 數位健康・Tech Lead
經歷	軟體工程師，投入開發十餘年
書	《AI 驅動開發：從 Prompt 到 Loop》





DevOpsDays Taipei 2026



AAday 2026

我講過的場次

HWDC 2025
不是用 AI，而是和 AI 一起工作：一場來自開發現場的 AI 流程導入

JSDC 2025
打造多視窗跨平台應用：一個基於 Electron + React 的企業實戰案例

DevOpsDays Taipei 2026
從寫到管：打造 GCP DevOps Agent

AAday 2026
從一張 Jira 工單，到一個可審查 PR

COSCUP 2026
我把自己寫的開源 SDD 工具丟到 15 個專案實測

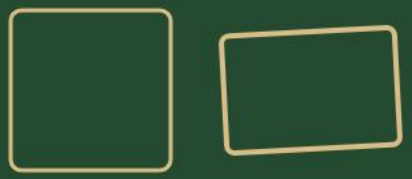
HWDC 2026（今天）
產出的人不能驗自己：AI 時代的驗收設計

Claude Code 官方 best practices

「看起來做完了」是唯一的訊號。

"looks done" is the only signal available, and you become the verification loop.

code.claude.com/docs/en/best-practices



3 /23

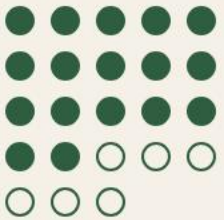
寫 code 的同一個 session 自己審



同一批問題

17 /23

全新的 reviewer



業界個人實驗：2 個 PR、12 次 review、單一作者

出處：@tvykruta 公開實驗（單一作者、2 個 PR）



我自己的專案

它說都測過了。
審查者列了五件，先看兩件。

漏掉的	同一個使用者再次進入（App 重新交握），已經勾好的內容會被清掉。
說了沒做的	文件寫「換人重置已測過」；把那段程式拿掉，測試照樣全過。
之後	實作的 agent 修好、補上測試，再派一個沒看過過程的 agent 複驗。

審查列了 5 個中嚴重度問題，這頁挑 2 個。

今天只講一件事

SDD 先講好怎樣算做完 Harness 讓 AI 交出可核對證據

Harness 做得好，你才敢放手。



SDD：先寫好怎樣算對，再讓 AI 做

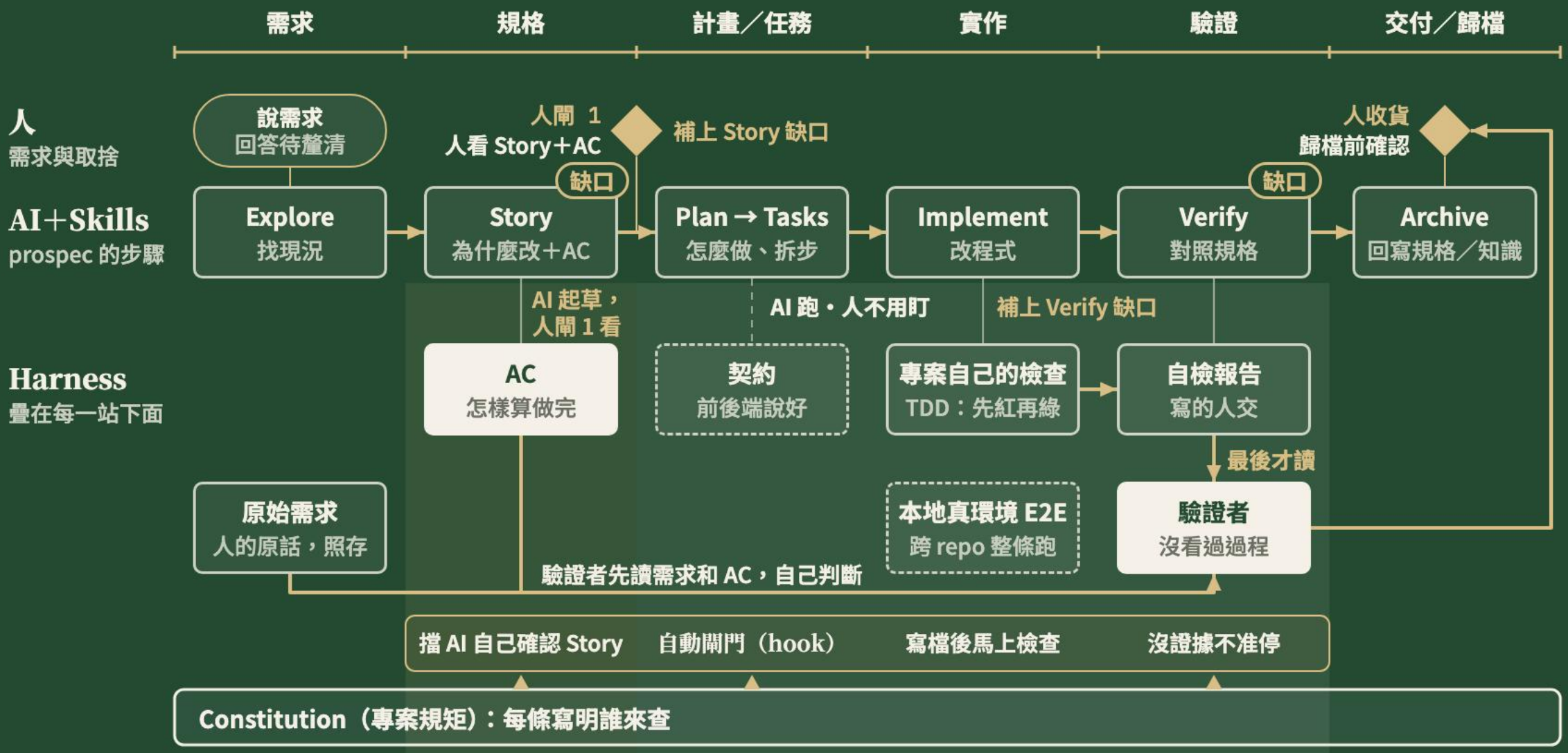


這裡，等一下疊上 Harness

兩個缺口：Story 寫完不一定停；Verify 只對照規格，不跑測試。

全景：SDD × Harness

人定好做完、收貨；中間交給 AI 跑



實線框：在用　虛線框：還在試　◆人閘　銅金箭頭：交出去的東西　灰細線：這站的 Harness　▲這站對照規矩　亮底：人不用盯



Harness：讓 AI 交出可核對證據的整套東西

規矩、檢查、驗證者、報告。

> 我現在的分工：Claude Code 實作；
> 驗證交給一個沒看過過程的 agent。

今天只用 5 個詞

AC
怎樣算做完

Constitution
專案規矩

契約
前後端說好的格式

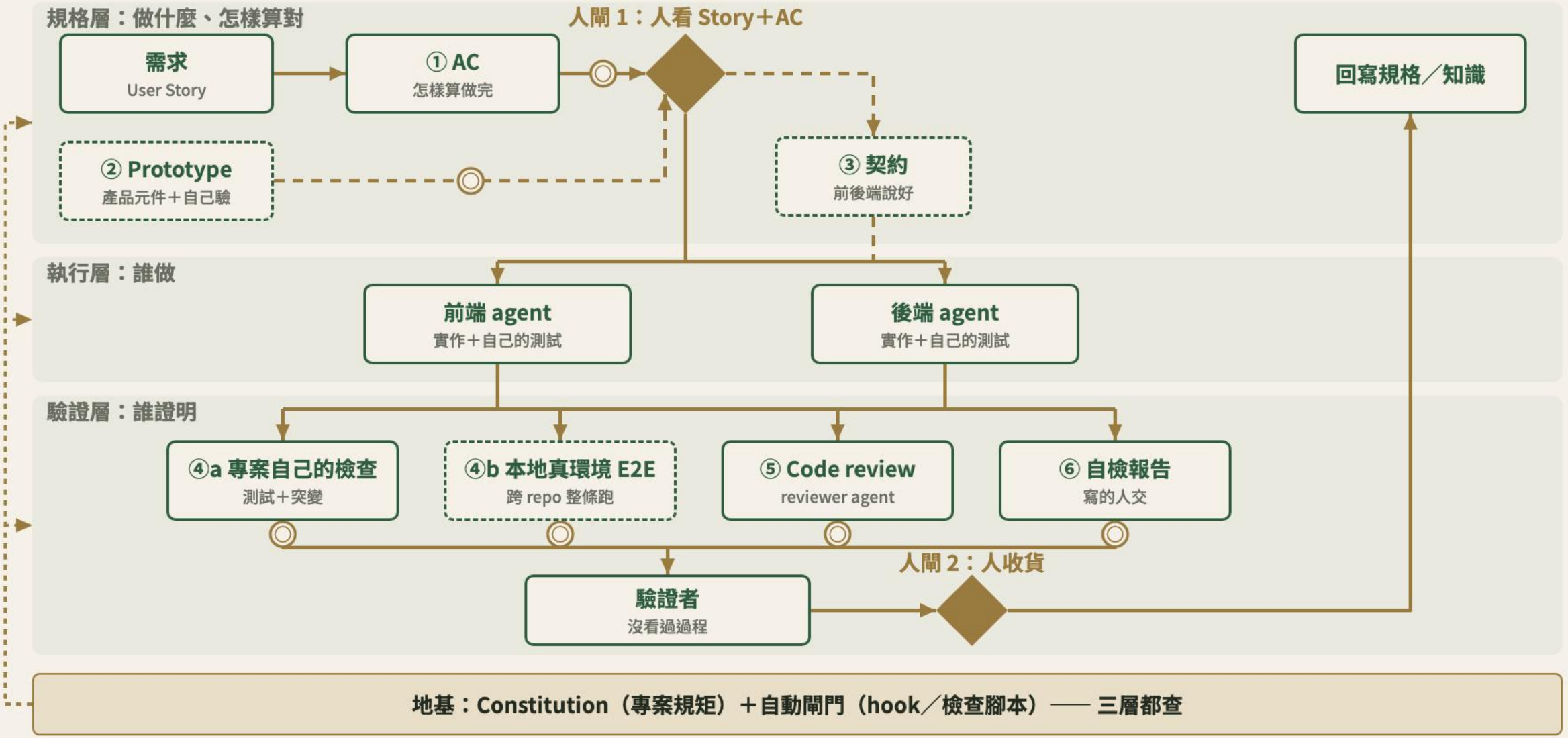
驗證者
沒看過過程的檢查員

自檢報告
交貨單

今天的地圖

三層 × 兩道人閘 × 一塊地基

這是我現在的整理，每個節點今天都會回來。



實線框：在用　虛線框：還在試（②③④b）　菱形：人閘　小圓章：不是寫的人在驗（6 枚）　銅金虛線：地基查三層

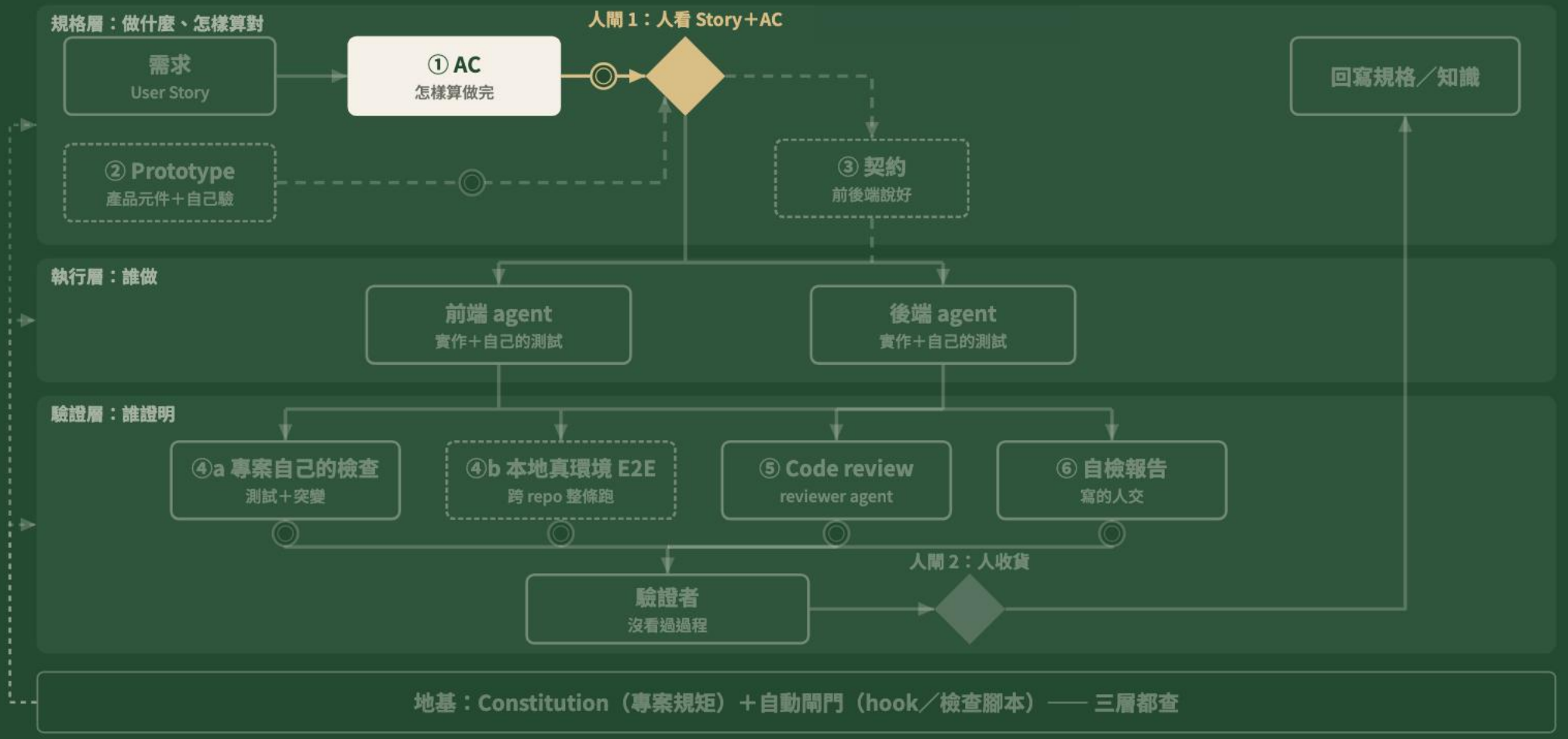
每段結尾，我會問一次

這段，
誰不能驗自己？



①

AC：怎樣算做完


規格層：做什麼、怎樣算對
人關 1：人看 Story+AC
需求
User Story
① AC
怎樣算做完
回寫規格／知識
② Prototype
產品元件＋自己驗
③ 契約
前後端說好
執行層：誰做
前端 agent
實作＋自己的測試
後端 agent
實作＋自己的測試
驗證層：誰證明
④a 專案自己的檢查
測試＋突變
④b 本地真環境 E2E
跨 repo 整條跑
⑤ Code review
reviewer agent
⑥ 自檢報告
寫的人交
人關 2：人收貨
驗證者
沒看過過程
地基：Constitution（專案規矩）＋自動閘門（hook／檢查腳本）—— 三層都查

User Story 長這樣

> As a 每天使用 App 回報狀況的使用者
>
> I want 每天勾選今天的狀況並送出，必要時寫備註
>
> So that 團隊每天都拿得到我的紀錄
>
> 優先度：P1
>
> Independent Test：用一個今天還沒回報的測試帳號，從頭跑一次

壞

送出要有防呆。

好

WHEN 一項都沒勾就按送出

THEN 無法送出，今天仍然是「未回報」

怎麼驗：機器驗，確認沒有發出任何寫入請求

WHEN／THEN＋具體值＋怎麼驗



寫完 AC，問三個問題

有邊界、看得到、寫得出怎麼驗

有邊界	寫具體值，剛好等於門檻那一格也要寫。
看得到	結果從畫面、API 或資料看得到，不是「內部要正確」。
怎麼驗	附一個指令或一件要看的東西，標「機器驗」或「人看」。

AC → 檢查

先看到紅，再做到綠



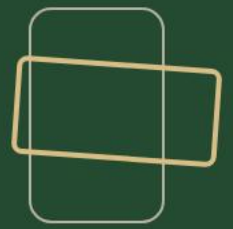
問自己：換一個合理但錯的寫法，這個測試還會過嗎？程式本來就對時，改壞一處，測試要變紅。

不知道，就寫不知道

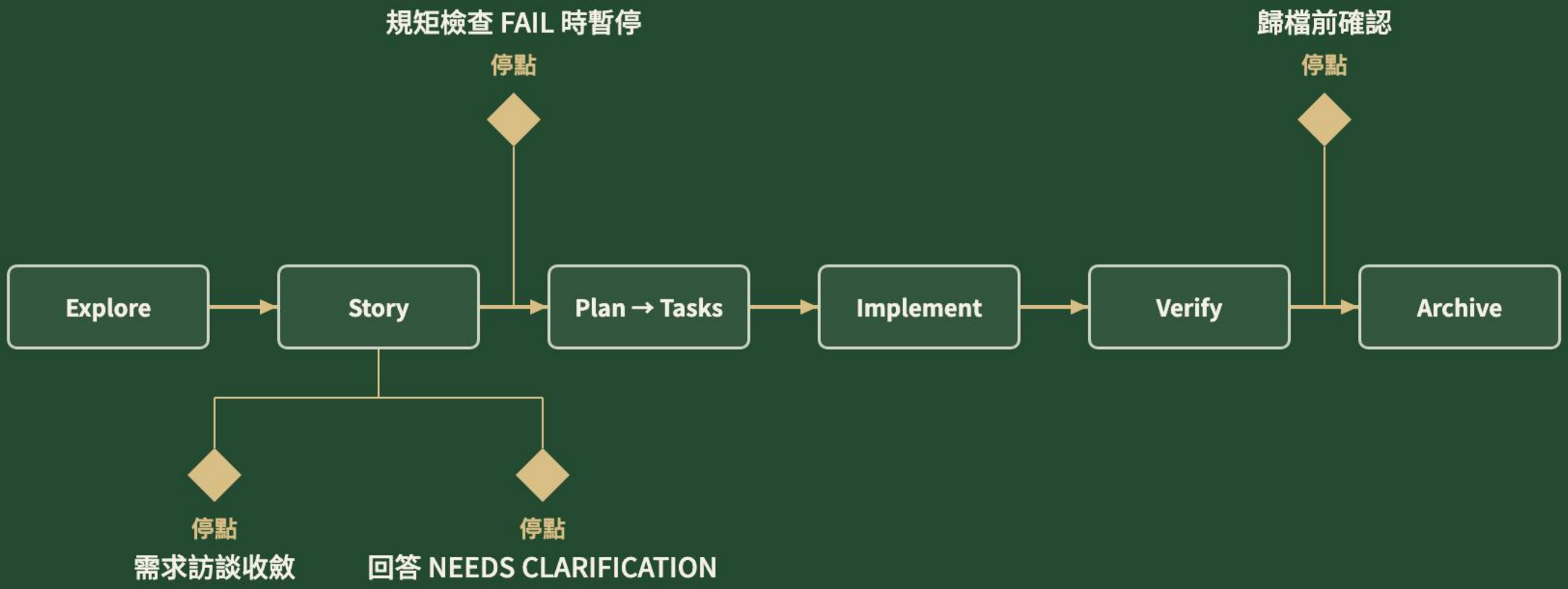
> 只填備註、一項都沒勾：依決定字面應該擋下
>
> → `NEEDS CLARIFICATION`：要請負責人確認
>
> 送出失敗（例如網路中斷）：prototype 沒有後端，沒有失敗狀態可參考
>
> → `NEEDS CLARIFICATION`

以人的決定為準，
不以 prototype 為準。

寫在 AC 旁邊，AI 才不會拿錯的東西當證據。



prospec 原本的停點



錯了收得回來，才放手



反方

Kiro 把一個小 bug 展成 4 個 Story、16 條 AC。

「I'd rather review code than all these markdown files.」

Birgitta Böckeler, martinfowler.com, 2025-10-15

我的做法

一個 Story 的 AC 最多 7 條。人看得完，才是最便宜的看點。



Constitution：
專案不可協商的幾條規矩

原則、為什麼、怎麼修改。最多 7 條，先用 5 條預設。

> GitHub Spec Kit 的 constitution：測試要先寫，而且要先失敗。

沒寫誰來查的規矩，只是願望

原則 主張需要來源

查不到來源，就寫 `not_found`，不准編造。

Enforcement 研究輸出必須帶來源網址、查詢日期、對應的主張、不確定程度。

這段，誰不能驗自己？

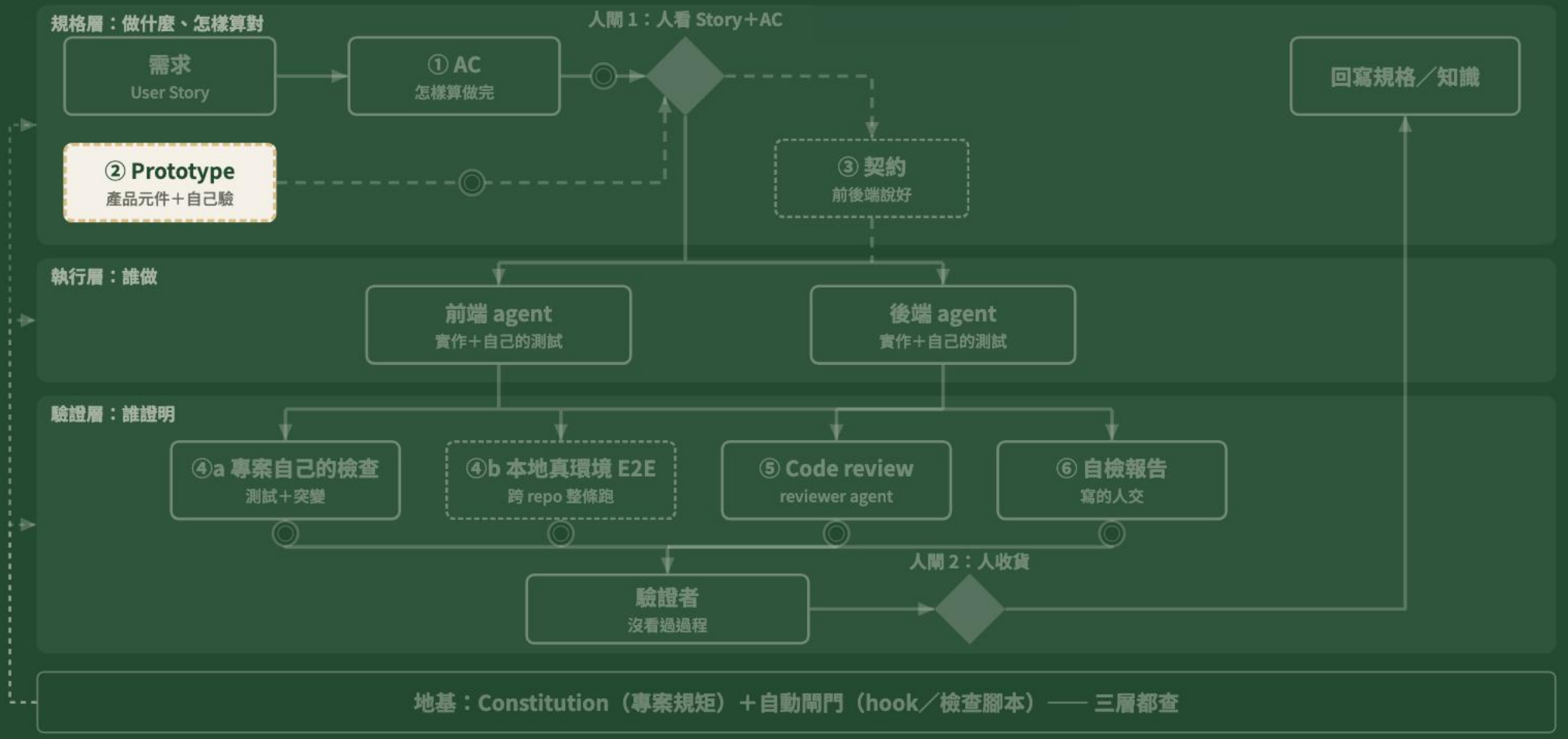
寫 AC 的 AI。

AC 交給人看，
規矩交給閘門查。



②

prototype 跟產品的落差





我們遇到的狀況

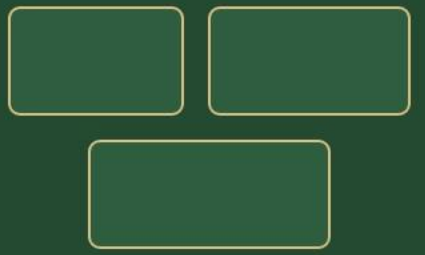
prototype 很像產品，但不是產品

誰做的	需求端用 AI 工具做 prototype
問題	同一份 prototype 裡，不同頁的規則互相矛盾

規則給方向，
實體元件給答案。

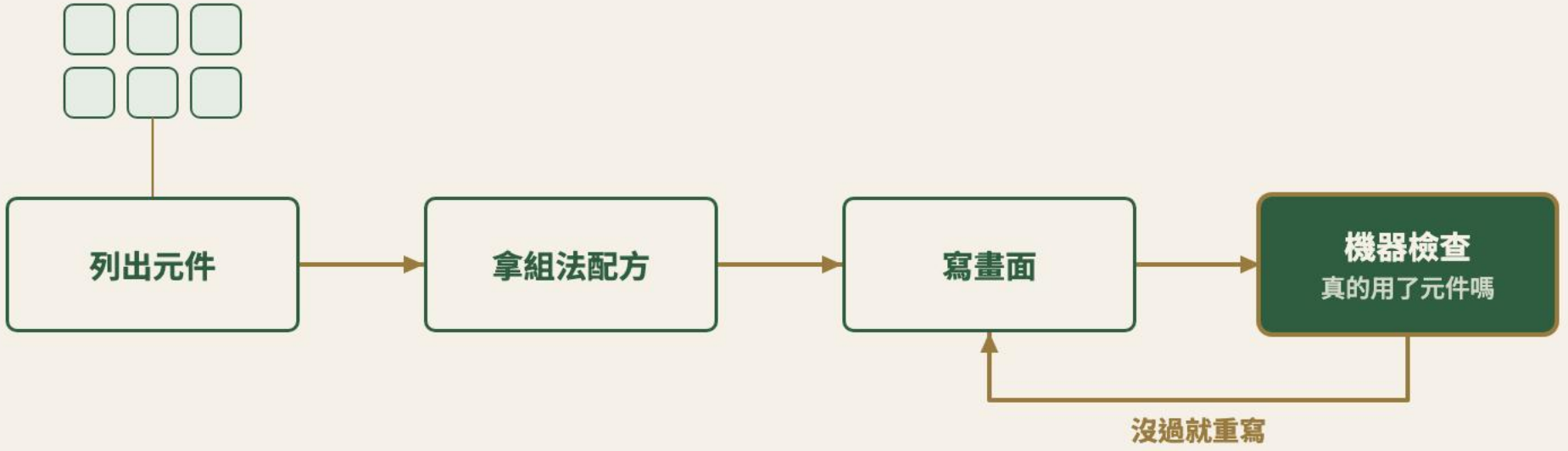
給 AI 規則，它照著畫；給 AI 元件，它直接拿來用。





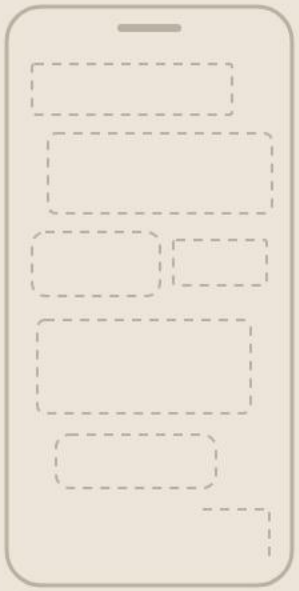
Storybook × MCP

讓 AI 拿產品元件做 prototype



驗證那一步是機器檢查（verify 工具），不是 AI 說了算。

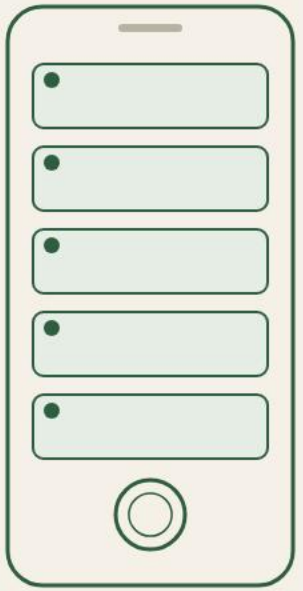
沒接元件庫



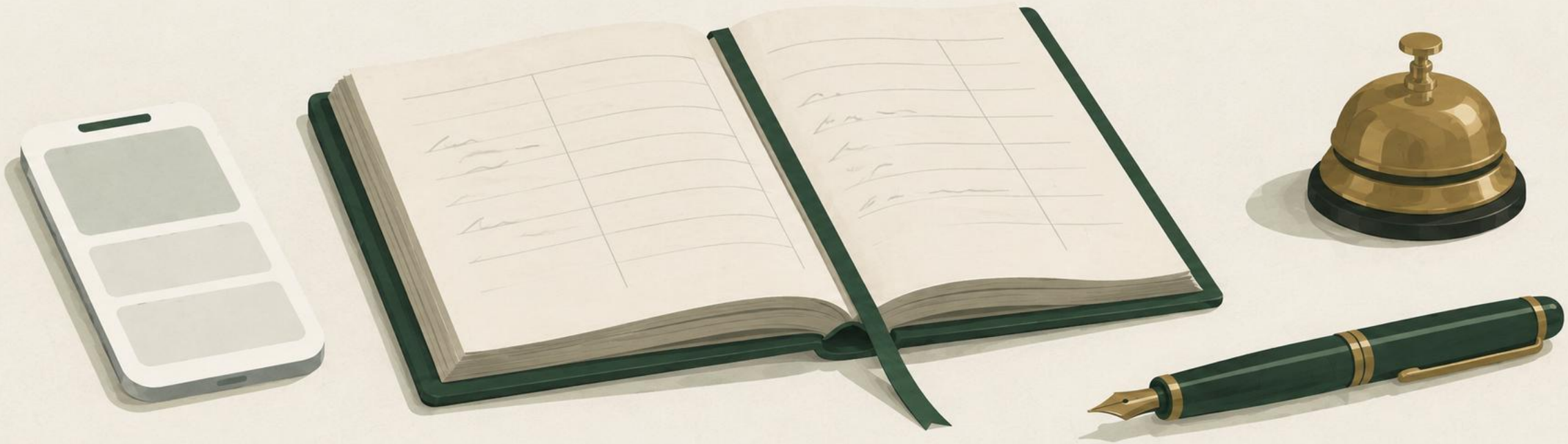
元件自己拼
狀態全缺
欄位名自己取

工程端對照
同一個模型
同一句需求

接上元件庫



全部用元件庫的元件
機器檢查通過
（verify 工具）



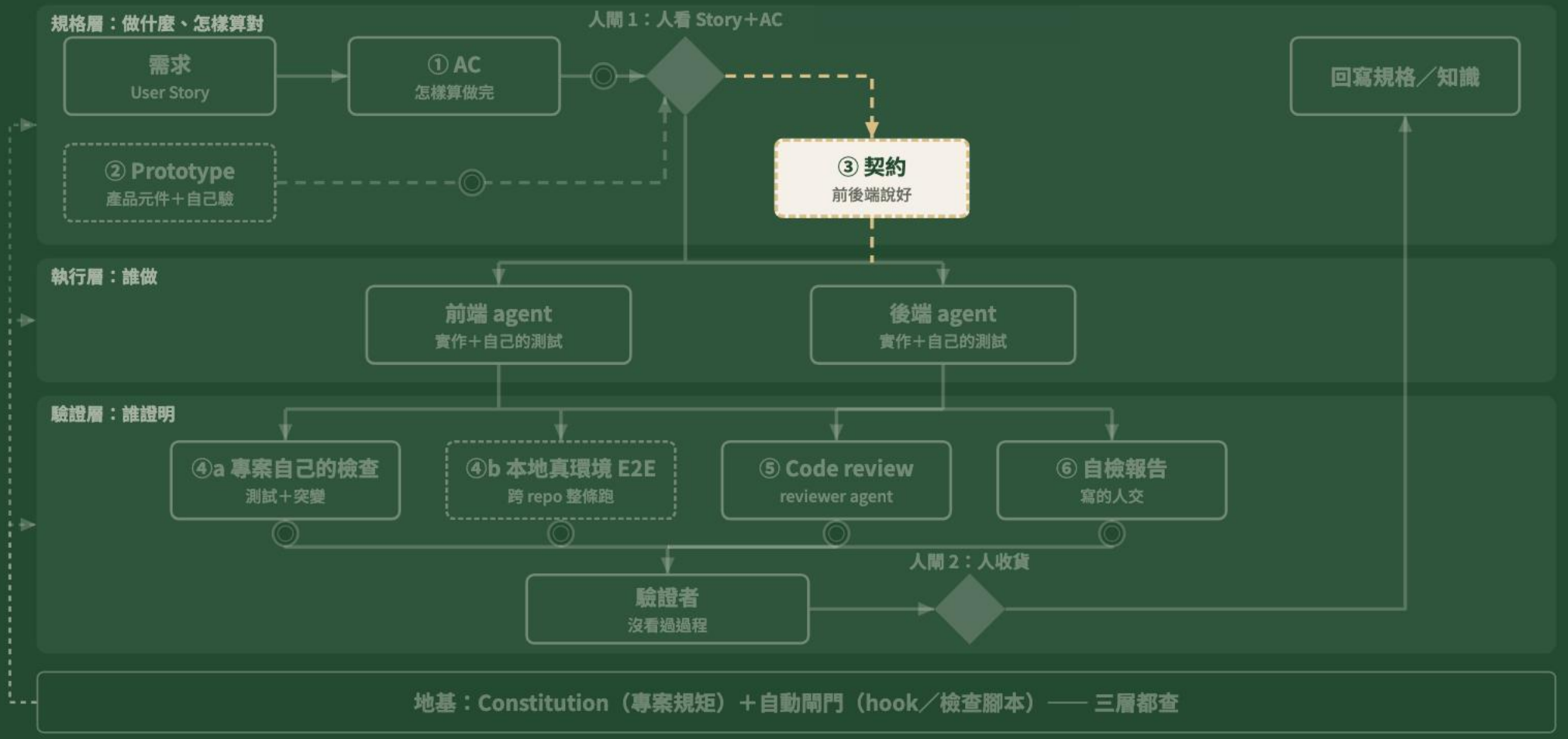
AI 盤點差異，人拍板。

每條差異標證據：〔原始碼〕〔截圖〕〔推論〕〔查不到〕。
prototype 只決定位置與版型。

有時產品改成照 prototype，有時改 prototype。
決定寫進決策紀錄。

③

契約：前後端說好的格式





同一個需求，
兩個 repo，兩個 agent。

各自做對不難，難的是兩邊對得上。

> 一個功能的識別字串，要在 9 個地方逐字一致。

規格集中，實作分開


規格專案
只產出規格
前端 repo
story → plan → tasks → 實作
契約
後端 repo
story → plan → tasks → 實作＋契約

契約寫兩邊會吵架的地方

- owner／身分
- 時區與時段
- 授權拒絕
- 資料保留
- 欄位
- 狀態轉移
- 錯誤與重試
- 成功後怎麼重讀



契約草案的標記

每一條，標它查到哪裡

- 〔實讀〕 有人讀過那一行程式碼
- 〔推論〕 從讀碼推出來，沒實際跑
- 〔未驗證〕 讀不到，或沒查

圖中：● 深綠封蠟　● 銅金封蠟　● 灰綠封蠟

「沒有任何一條行為斷言是實跑過的。」

驗收分兩層

上層

規格 AC → E2E 情境

下層

各 repo 自己的測試 → 故意改壞，看會不會變紅

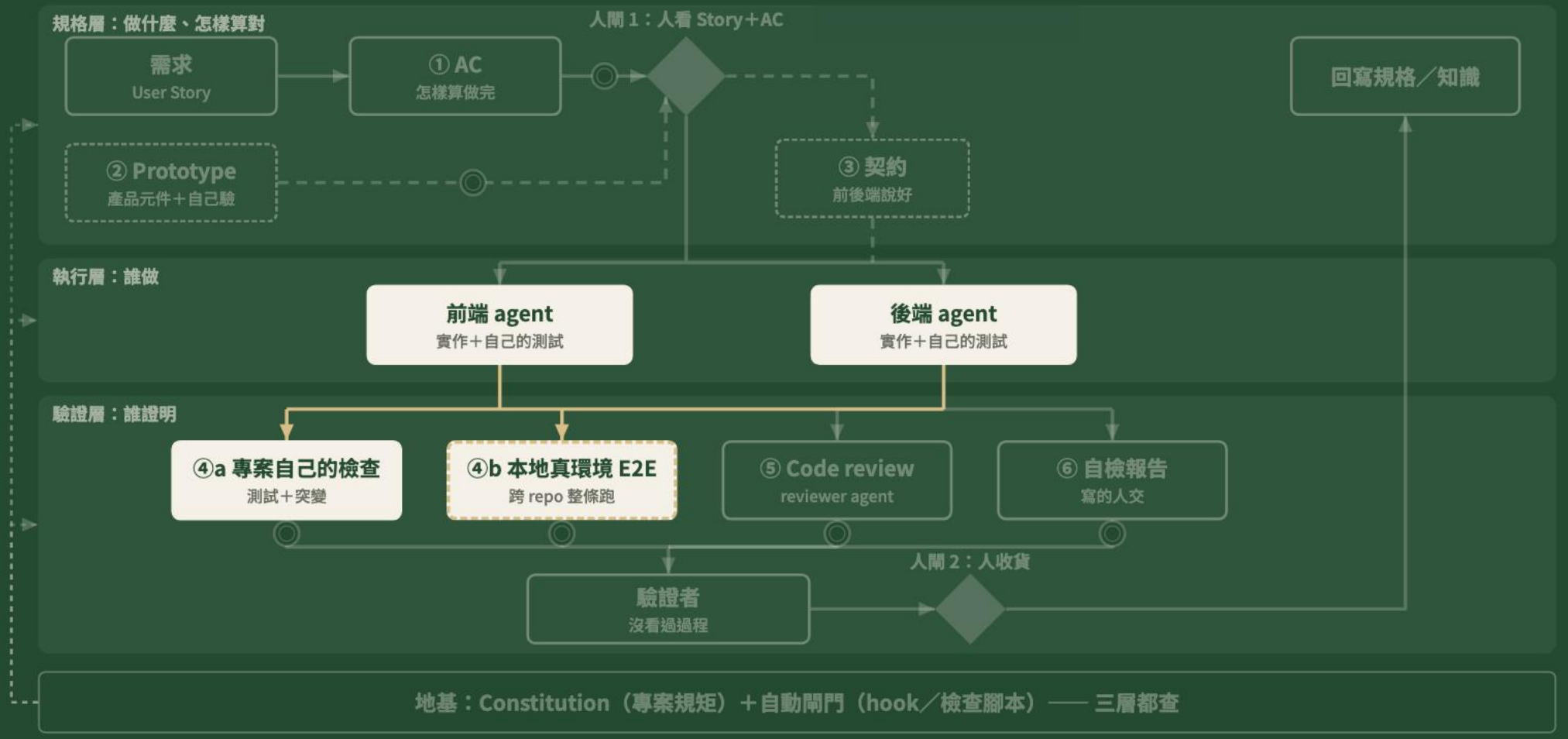
24 個突變，

23 個轉紅

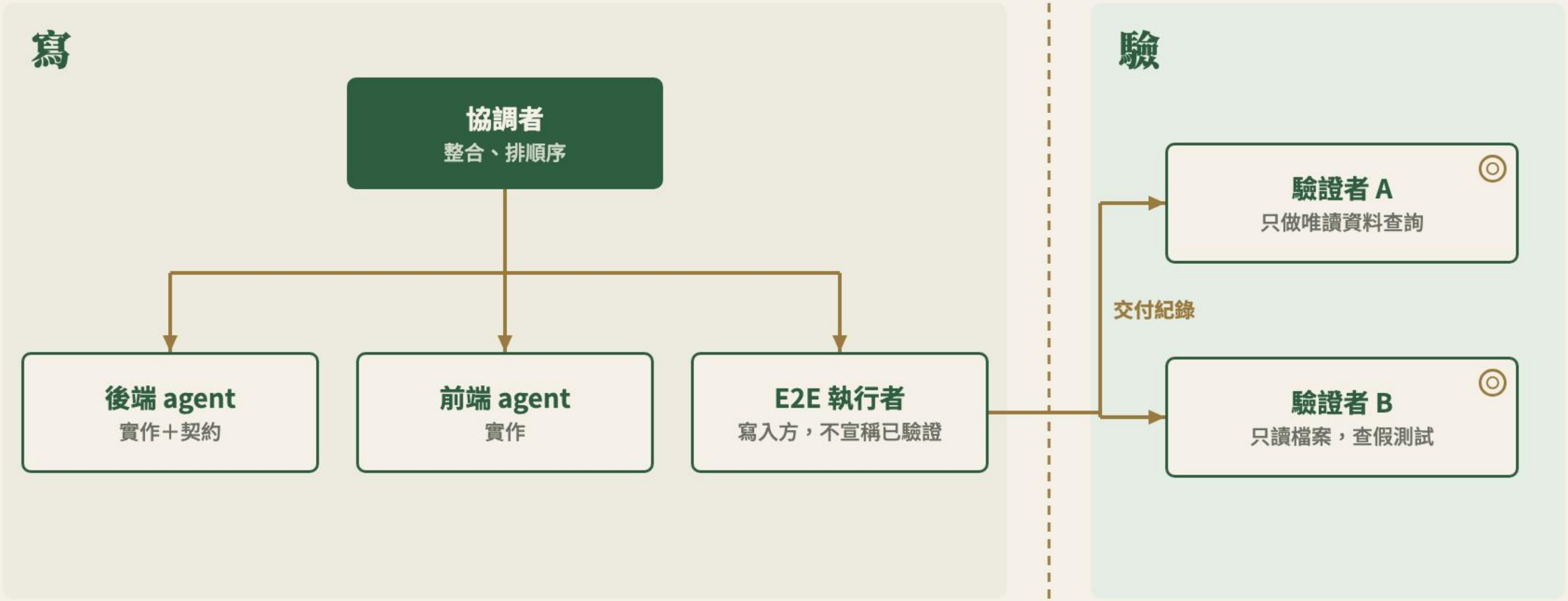


④

分工，跟整條跑一次


規格層：做什麼、怎樣算對
人閘 1：人看 Story+AC
需求
User Story
① AC
怎樣算做完
回寫規格／知識
② Prototype
產品元件＋自己驗
③ 契約
前後端說好
執行層：誰做
前端 agent
實作＋自己的測試
後端 agent
實作＋自己的測試
驗證層：誰證明
④a 專案自己的檢查
測試＋突變
④b 本地真環境 E2E
跨 repo 整條跑
⑤ Code review
reviewer agent
⑥ 自檢報告
寫的人交
人閘 2：人收貨
驗證者
沒看過過程
地基：Constitution（專案規矩）＋自動閘門（hook／檢查腳本）—— 三層都查

寫的跟驗的，分開


寫
協調者
整合、排順序
後端 agent
實作＋契約
前端 agent
實作
E2E 執行者
寫入方，不宣稱已驗證
驗
驗證者 A
只做唯讀資料查詢
交付紀錄
驗證者 B
只讀檔案，查假測試

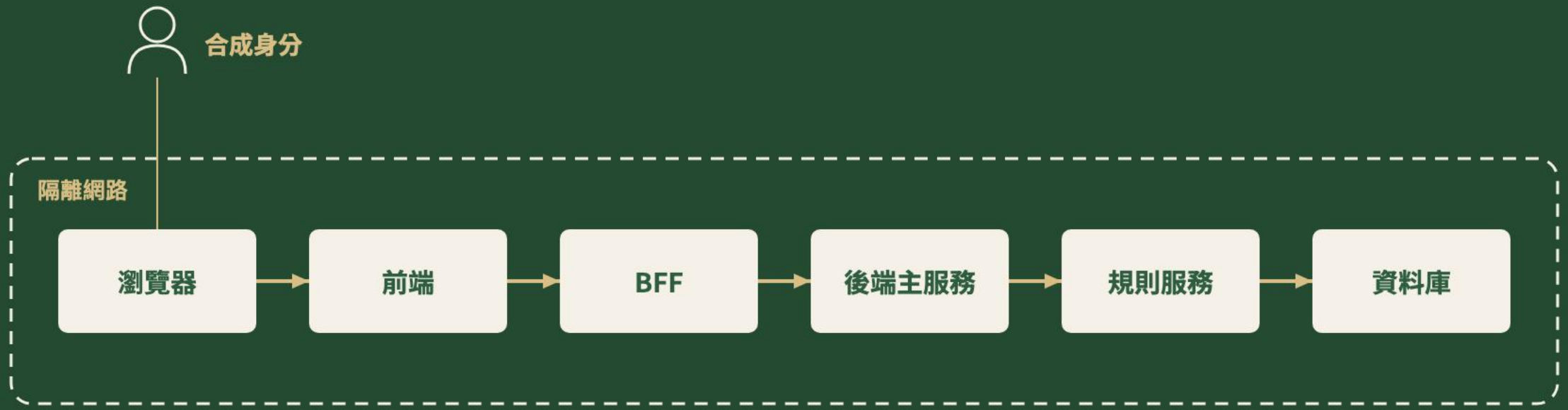


派工單

只寫三件事

擁有	哪些檔案、哪個 worktree
輸入	照哪一份契約
完成證據	交什麼才算做完

真服務，隔離網路，合成身分



全部在隔離網路裡；只匯入必要的表與合成身分，不複製任何使用者資料。

E2E 的兩條規矩

1. 非本機的請求，一律擋下。
2. 要證明「拒絕」，先證明兩個合法身分各自能成功；

   500、空回應、失效憑證，都不算擋住。



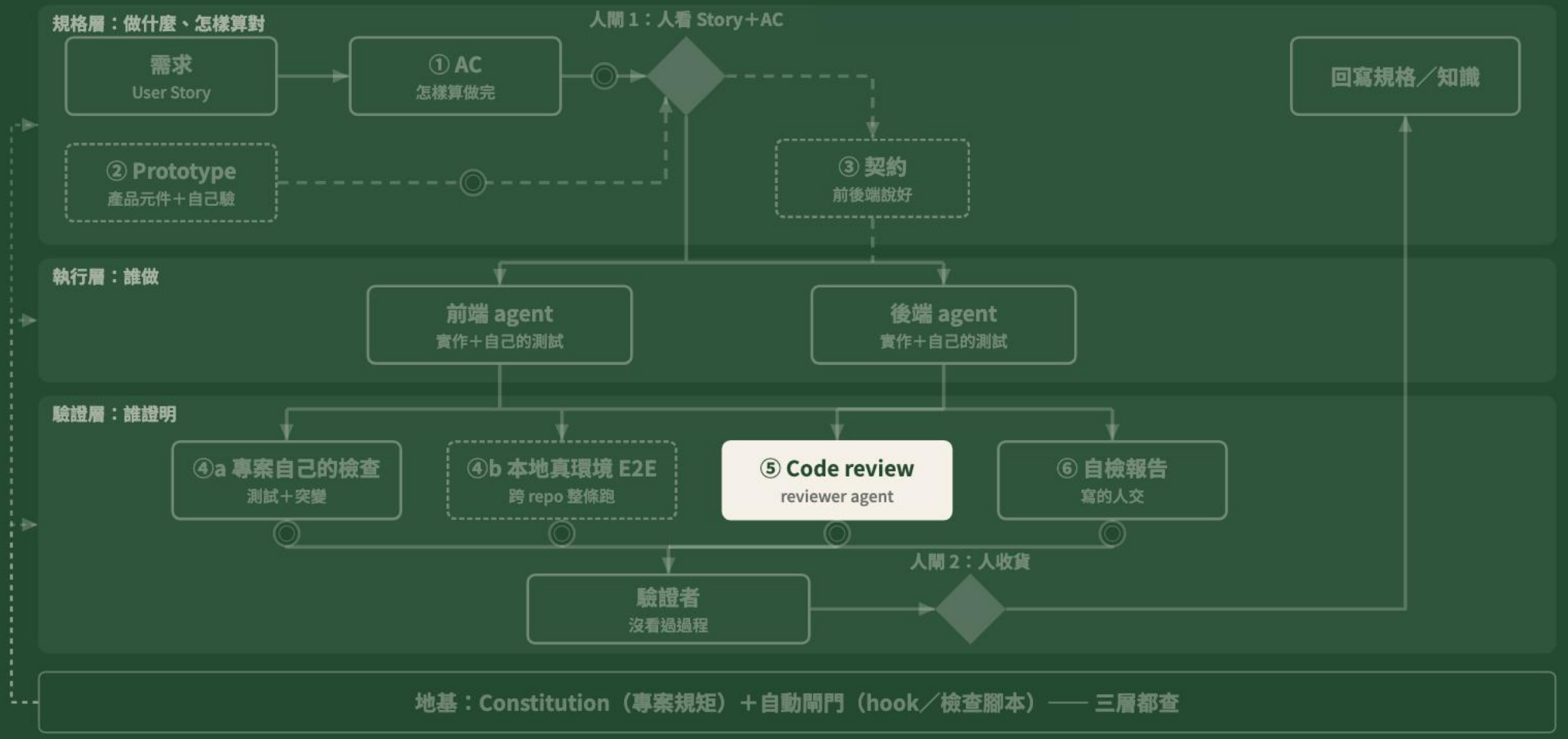
4 個情境首跑 FAIL

錯的是測試自己寫的預期。

不重跑（重跑會重複寫入）。用首跑的實際值重判，兩份結果都留，交給驗證者判。

⑤

Code review agent



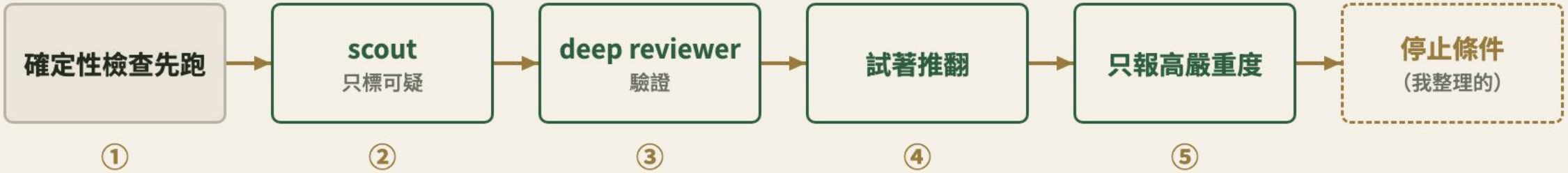
我查了一圈公開案例

精準是一條管線，不是一個 prompt。

沒有一家把精準歸功於換更強的模型。



業界各家的五個機制，我把它們排成一條







Matt Pocock《Fixing the PR Bottleneck》

規範放 reviewer，不放實作者

04:49	"But checks can lie."
11:36	"Don't put coding standards in your implementer agent."

我的 review：實跑，而且說得出怎麼知道的



候選說

這裡拿不到時區，程式~~會當掉~~。

你怎麼知道的？

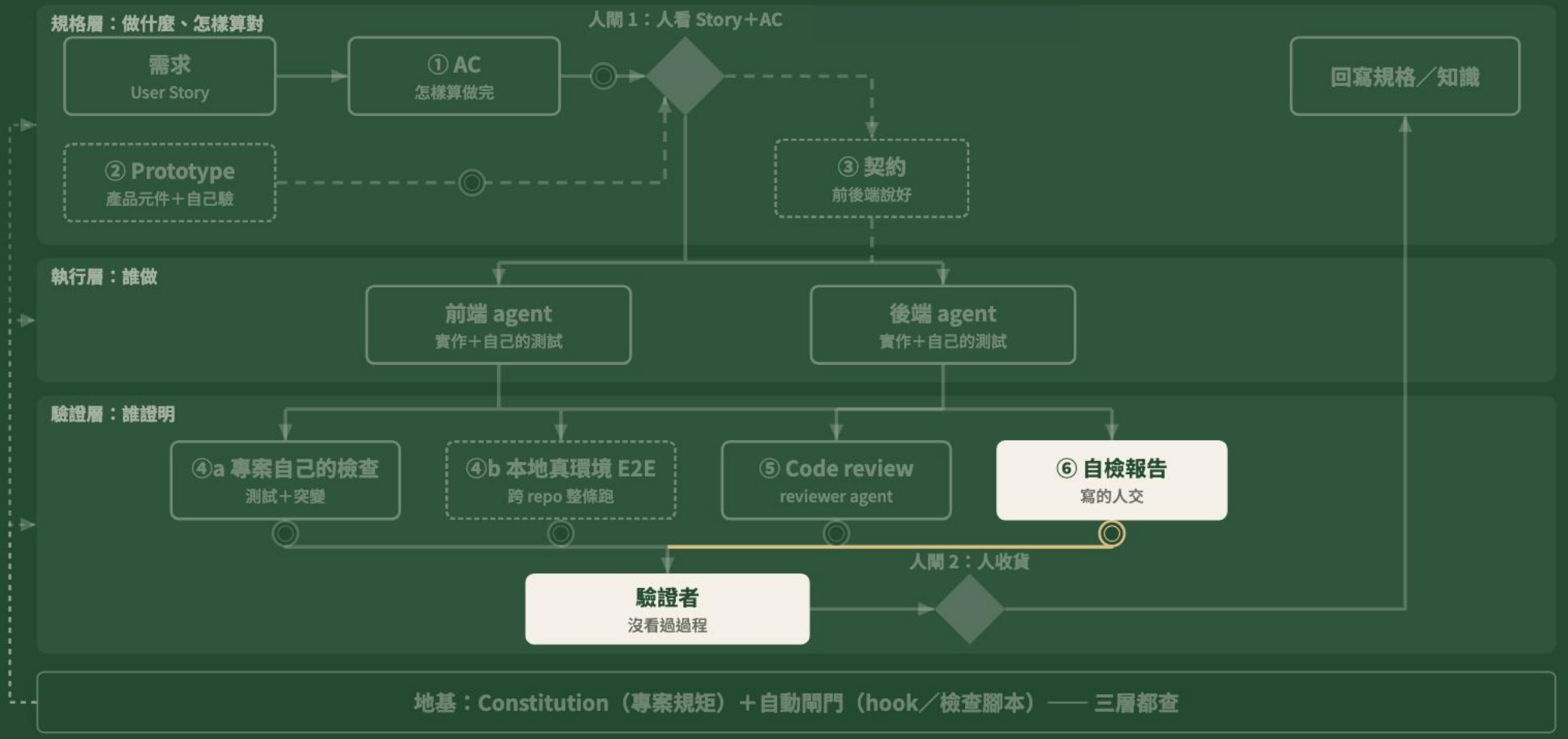
驗證者追到的

拿不到時區時，會退回預設時區，
不會當掉。這條在送出前被拿掉。

證據不夠的發現，不送出去。

⑥

自檢報告：交貨單


規格層：做什麼、怎樣算對
人閘 1：人看 Story+AC
需求
User Story
① AC
怎樣算做完
回寫規格／知識
② Prototype
產品元件＋自己驗
③ 契約
前後端說好
執行層：誰做
前端 agent
實作＋自己的測試
後端 agent
實作＋自己的測試
驗證層：誰證明
④a 專案自己的檢查
測試＋突變
④b 本地真環境 E2E
跨 repo 整條跑
⑤ Code review
reviewer agent
⑥ 自檢報告
寫的人交
人閘 2：人收貨
驗證者
沒看過過程
地基：Constitution（專案規矩）＋自動閘門（hook／檢查腳本）—— 三層都查

標題就寫：尚未經獨立核對

寫入方紀錄，尚未經獨立核對 未核對

沒做的事：沒 commit、沒 push、沒開 MR、沒碰正式環境

1. 受測版本與環境
2. 哪些是真的、哪些是 stub
3. 指令與 exit code（失敗照寫）
4. 結果：首跑原樣、重判方式
5. 留紅與未驗
6. 與計畫不同的地方

常見的回報

「應該都好了！」

自檢報告（示意）

AC-1	PASS	來源｜指令｜結果｜先紅
AC-2	PASS	來源｜指令｜結果｜既有滿足・突變
AC-3	待人看	要人看什麼、在哪裡看

示意，不是真實報告

有一次，AI 自報所有閘門全綠

留紅不算失敗，湊出來的綠才算。

全綠的那份，藏著一個會覆寫使用者手寫 CLAUDE.md 的 bug。



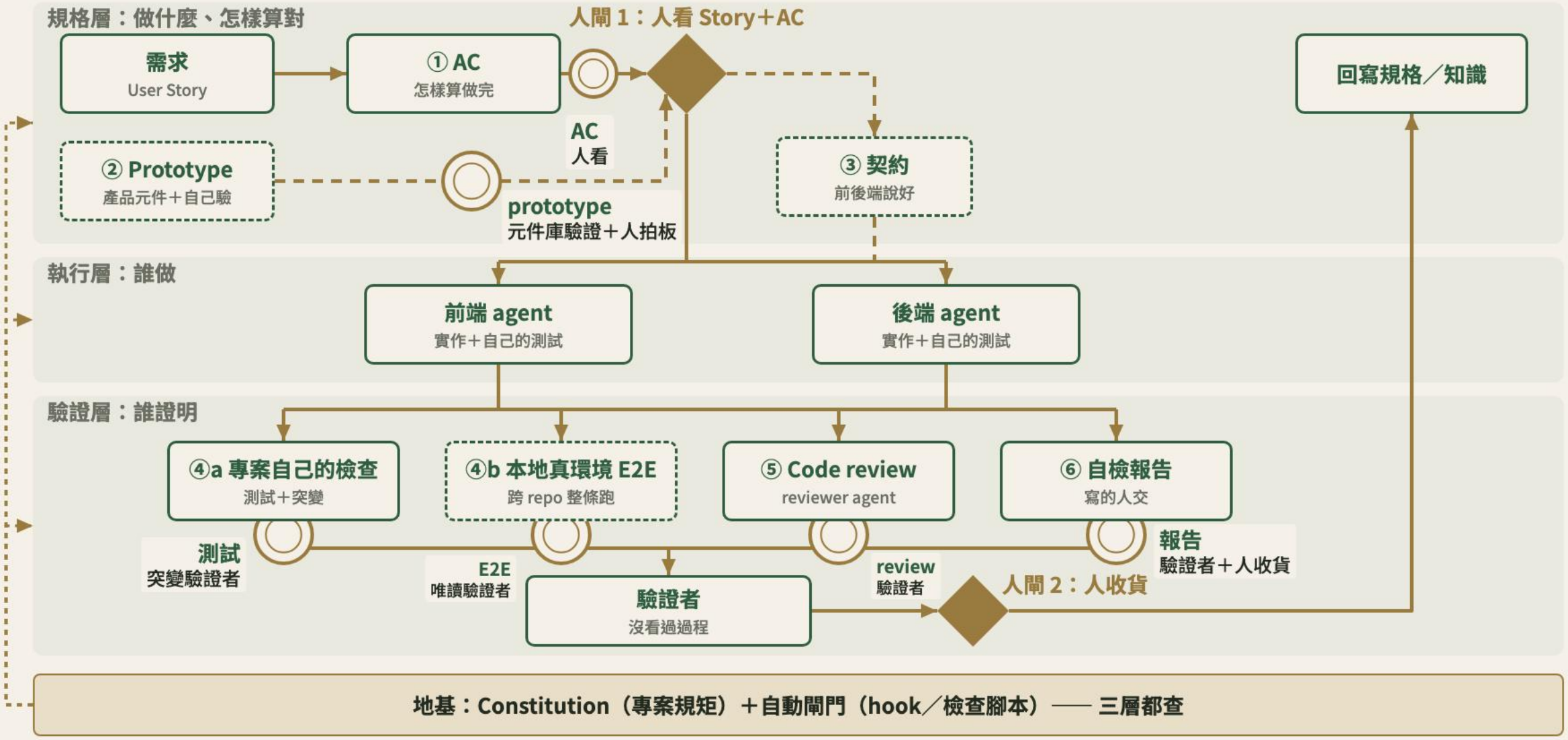
報告交給驗證者，不是交給你



整條責任鏈

每個交接，都安排了一個不是寫的人在驗

6 枚印章：①②④a④b⑤⑥
③ 契約：我們正在試



實線框：在用　虛線框：還在試（②③④b）　◆ 菱形：人閘　◎ 小圓章：不是寫的人在驗（6 枚）　銅金虛線：地基查三層



放手到什麼程度

設好邊界，就能放手

邊界	所有決定都標「暫定」； 不 push、不開 MR、不動工單
驗證	每一段都交給沒看過過程的驗證者
早上	我看交接，改掉不同意的暫定決定

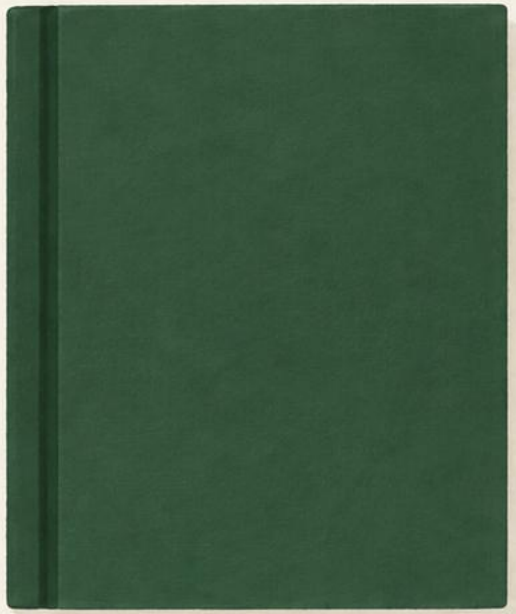
回去怎麼套用

裝這個包，記三個入口

讓 AI 按驗收條件開發，交出可核對的證據。

```
claude plugin marketplace add ci-yang/sdd-harness
claude plugin install sdd-harness@sdd-harness
```

`/sdd-harness:start`	建好 sdd/、預設規矩，跑健檢
`/sdd-harness:story`	存需求原文、寫 AC，停下來等你確認
`/sdd-harness:done`	跑檢查 → 自檢報告 → 驗證者 → 兩個結果







第一週，只做三件事

1 /start：裝好，跑健檢，規矩先用預設。
2 /story：寫一條 AC，你親自寫一個邊界例子、一個遺漏，再確認。
3 /done：看兩個結果，親自看抽到的那一條。

> 不用一次做完整張圖。

Harness 做得好，才能放手給 AI。

楊驒 Ci　謝謝

第一個問題送《AI 驅動開發》一本・其他問題歡迎到外面小桌



帶走

Harness 包

github.com/ci-yang/sdd-harness

prospec（開源）

github.com/ci-yang/prospec

書（天瓏）

tenlong.com.tw/products/9786267889688

Discord 社群

discord.com/invite/SFXK37uDgt

Threads

threads.net/@ci.fullstack

工作坊

15:30–17:30

今天 · 603＋604 會議室